

High Level Design

System architecture, flows and design decisions

Research Assistant - local-first multi-agent research workspace
Generated from 10-hld.md

10. High Level Design

Table of contents

1. Problem statement
2. Goals and non-goals
3. Requirements summary
4. System context
5. Containers
6. Major components
7. End-to-end flows
8. Data architecture
9. Key design decisions and trade-offs
10. Scalability, performance, reliability, security, and privacy
11. Capacity and latency characteristics
12. Design vs. implementation

1. Problem statement

Researchers need a way to discover papers, save the relevant ones, read and annotate them, and ask questions that stay grounded in the saved evidence. Generic chat systems are fast but often fabricate citations. Conventional reference managers store papers but do not verify answers. This application bridges those needs by making each research space a local evidence set and forcing answers through an auditable peer-review loop.

2. Goals and non-goals

2.1 Goals

- Support scholarly discovery across multiple public sources. Related requirements: FR-1.1, FR-1.2, FR-1.3, FR-1.8, FR-1.9.
- Let users pin papers and web captures into persistent research spaces. Related requirements: FR-2.
- Provide grounded Q&A with citations and a visible verification trail. Related requirements: FR-3 and LC-1 through LC-12.
- Provide compare and gap analysis over selected pinned papers. Related requirements: FR-4.
- Provide notes, auto notes, anchored annotations, memory, and content search. Related requirements: FR-5 and FR-6.
- Keep inference local through Ollama and minimize external data egress. Related requirements: NFR-3 and NFR-6.

2.2 Non-goals

- No authentication or multi-tenant authorization in the delivered app.
- No production Postgres deployment or Alembic migration chain.
- No Celery worker queue, LangGraph orchestration, or durable distributed graph state.
- No required Qdrant or GROBID service in the working path.
- No backend scraping of Google Scholar.

3. Requirements summary

Area	Implemented status
FR-1 Discovery	Implemented through adapters for arXiv, OpenAlex, Crossref, PubMed, Semantic Scholar, CORE, and optional web. Domain tags bias routing and ranking.
FR-2 Paper ingestion	Implemented for pinned records, PDFs, abstracts, and web captures. Chunks are persisted with embeddings in SQLite and loaded into an in-memory vector store.
FR-3 Grounded Q&A	Implemented through QAOrchestrator , HybridRetriever , AnswerAgent , PeerReviewerAgent , and VerificationLoopDriver .
FR-4 Compare and gap	Implemented through paper profiles, compare and gap agents, cached comparison_reports , and verification of conclusions.
FR-5 Notes	Implemented for manual notes, auto notes, anchored highlights, reader annotations, CRUD, and note context augmentation.
FR-6 Memory	Implemented as rolling summary, findings, open questions, recent turns, and recall in MemoryService .
NFR-1 Performance	Bounded by hard per-source deadlines, context budgets, and loop wall-clock caps. Local model latency dominates.
NFR-2 Accuracy	Verification loop enforces claim coverage, citation validity, and low-confidence warnings.
NFR-3 Privacy	LLM and embeddings use local Ollama. Discovery and web capture call external sources only for user-initiated searches and captures.
NFR-5 Reliability	Degrades per source, uses circuit breakers, SQLite retry on lock, and safe best-effort finalizers.
NFR-6 Security	URL validation rejects non-HTTPS, private, loopback, link-local, multicast, and reserved addresses; prompts delimit untrusted evidence.

4. System context

Diagram (mermaid source)

```

flowchart LR
  User[Researcher] --> UI[Next.js frontend]
  UI --> API[FastAPI backend]
  API --> Ollama[Local Ollama]
  API --> DB[(SQLite dev.db)]
  API --> Sources[Scholarly APIs]
  API --> Web[Web search provider]
  API -. optional health only .-> Qdrant[Qdrant]
  API -. configured but unused path .-> GROBID[GROBID]
  API -. future production option .-> Postgres[Postgres]
  Sources --> Arxiv[arXiv]
  Sources --> OpenAlex[OpenAlex]
  Sources --> Crossref[Crossref]

```

Sources --> PubMed[PubMed]
Sources --> S2[Semantic Scholar]
Sources --> CORE[CORE]

5. Container diagram

Diagram (mermaid source)

```
flowchart TB
    subgraph Browser
        FE[Next.js React app\nDashboard, SpaceWorkspace, reader, notes, compare]
    end
    subgraph Backend
        API[FastAPI routes under /v1]
        Services[Services\nDiscovery, Paper, Chat, Compare, Notes, Memory, Space]
        Verify[Agent and verification layer\nAnswerAgent, PeerReviewerAgent, VerificationLoopDriver]
        Retrieval[Retrieval layer\nHybridRetriever, SparseSearch, vector store, RRF, rerank, MMR]
        Sources[Source adapters and registry]
    end
    FE --> API
    API --> Services
    Services --> Verify
    Services --> Retrieval
    Services --> Sources
    Verify --> Ollama[Ollama models]
    Retrieval --> Ollama
    Services --> DB[(SQLite)]
    Retrieval --> DB
    Sources --> External[External APIs and web provider]
    Retrieval -. optional .-> Qdrant[Qdrant]
    Services -. optional future .-> Postgres[Postgres]
    Services -. optional future .-> GROBID[GROBID]
```

6. Major components

Component	Responsibility	Key files
Frontend dashboard	Create, rename, duplicate, archive, unarchive, delete spaces; display health.	frontend\components\dashboard.tsx
Space workspace	Main tabs for chat, library, compare, and notes; reader modal and command palette.	frontend\components\space-workspace.tsx
API router	Mounts admin, spaces, search, papers, chat, compare, notes, verification routes.	backend\app\api\router.py
Discovery service	Plan, fan out, normalize, deduplicate, rank, and partition web results.	services\discovery_service.py
Source registry	Source ordering, circuit breaker use, hard per-source async timeout.	sources\registry.py
Paper service	Pin, ingest, web capture, chunk, embed, persist, unpin, status and content.	services\paper_service.py
Retrieval	Dense plus sparse search, RRF fusion, rerank, MMR, evidence packing.	retrieval*.py

Component	Responsibility	Key files
Chat service	Memory context, query rewrite, verification loop, persistence of turns, citations, traces.	<code>services\chat_service.py</code>
Verification loop	Bounded async loop for draft, review, route, refine, re-retrieve, accept, best-effort, abstain.	<code>verification\loop.py</code> , <code>verification\policies.py</code>
Compare service	Profiles, matrix, gaps, conclusion verification, cache by paper set.	<code>services\compare_service.py</code>
Notes service	Auto notes, manual notes, anchors, annotation retrieval, note chunks in retrieval.	<code>services\note_service.py</code>
Persistence	SQLAlchemy models with SQLite development database.	<code>models*.py</code> , <code>core\dependencies.py</code>

7. End-to-end flows

7.1 Discovery with domain tags and optional web results

Diagram (mermaid source)

```
sequenceDiagram
    participant U as User
    participant FE as Frontend
    participant API as Search API
    participant Planner as QueryPlanner
    participant Registry as SourceRegistry
    participant Adapters as Source adapters
    participant Rank as Deduplicator and RankingAgent
    U->>FE: Search with domain tags and optional web
    FE->>API: SearchQuery
    API->>Planner: plan(query, filters, tags, limit)
    Planner-->>API: SearchPlan with source_hints and query_terms
    API->>Registry: fan_out(plan)
    Registry->>Adapters: parallel search with hard SOURCE_TIMEOUT_S
    Adapters-->>Registry: RawPaperRecord lists plus degraded errors
    Registry-->>API: raw records and SourceHealth
    API->>Rank: normalize, dedupe, rank
    Rank-->>API: scholarly results and optional web_results
    API-->>FE: RankedPaperList
```

7.2 Paper pin, ingest, and embed

Diagram (mermaid source)

```
sequenceDiagram
    participant FE as Frontend
    participant API as Papers API
    participant Paper as PaperService
    participant DB as SQLite
    participant Net as PDF URL
    participant Ollama as Ollama embeddings
    participant Store as InMemoryVectorStore
    FE->>API: RawPaperRecord
    API->>Paper: pin_and_ingest(space_id, record)
    Paper->>DB: upsert paper by DOI, arXiv ID, content hash
```

```

Paper->>DB: ensure unique pin
alt existing chunks
  Paper->>DB: load chunks and embeddings
  Paper->>Store: upsert into space store
else new content
  Paper->>Net: safe HTTPS fetch and PDF magic-byte check
  Paper->>Paper: parse pages, chunk per page, detect section
  Paper->>Ollama: embed batches with nomic-embed-text
  Paper->>DB: persist chunks and embedding_json
  Paper->>Store: register chunks
end
Paper-->>FE: IngestJob with READY, DEGRADED, or FAILED

```

7.3 Grounded Q&A through verification loop

Diagram (mermaid source)

```

sequenceDiagram
    participant FE as ChatPanel
    participant API as Chat API
    participant QA as QAOrchestrator
    participant Mem as MemoryService
    participant Rewriter as QueryRewriter
    participant Ret as NotesAware HybridRetriever
    participant Ans as AnswerAgent
    participant Rev as PeerReviewerAgent
    participant Pol as LoopPolicies
    participant DB as SQLite
    FE->>API: question
    API->>QA: answer(space_id, question)
    QA->>Mem: get_context
    QA->>Rewriter: rewrite if referential
    QA->>Ret: retrieve evidence
    loop until accept, abstain, best effort, timeout
        QA->>Ans: generate draft with cited chunk ids
        QA->>Rev: review claims against evidence
        QA->>Pol: prepare_verdict and decide
        alt evidence gap
            QA->>Ret: re-retrieve with missing queries
        else regenerate
            QA->>Ans: next draft receives history
        end
    end
    QA->>DB: persist user turn, assistant turn, citations, verification_iterations
    API-->>FE: AnsweredTurn with citations, confidence, iterations, verified, verification_url

```

7.4 Compare and gap

Diagram (mermaid source)

```

sequenceDiagram
    participant FE as ComparePanel
    participant API as Compare API
    participant CS as CompareService
    participant Profiles as ProfileStore
    participant CA as CompareAgent
    participant GA as GapAgent
    participant Loop as VerificationLoopDriver
    participant DB as SQLite
    FE->>API: paper_ids, refresh
    API->>CS: compare(space_id, paper_ids)
    CS->>DB: validate space and pins
    CS->>DB: return newest cached report unless refresh
    CS->>Profiles: get_or_build for each paper
    CS->>CA: synthesize comparison matrix
    CS->>GA: identify gaps
    CS->>Loop: verify comparison conclusion over profile evidence
    CS->>DB: persist comparison_report
    API-->>FE: ComparisonReportDTO

```

7.5 Web capture

Diagram (mermaid source)

```
sequenceDiagram
    participant FE as Frontend
    participant API as Web capture API
    participant Paper as PaperService
    participant Robots as robots.txt
    participant Web as HTTPS page
    participant Ollama as Embeddings
    participant DB as SQLite
    FE->>API: url, optional title, optional selected content
    API->>Paper: capture_web
    Paper->>Paper: validate public HTTPS URL
    alt selected content supplied
        Paper->>Paper: skip network fetch
    else full page
        Paper->>Robots: check can_fetch
        Paper->>Web: stream with 5 MB cap and redirect validation
        Paper->>Paper: parse PDF or readable HTML
    end
    Paper->>DB: upsert web paper and pin
    Paper->>Ollama: embed chunks
    Paper->>DB: persist chunks
    API-->>FE: IngestJob
```

7.6 Annotation

Diagram (mermaid source)

```
sequenceDiagram
    participant FE as PaperReader
    participant API as Notes routes
    participant NS as NoteService
    participant DB as SQLite
    FE->>API: GET paper content
    API-->>FE: chunks ordered by ordinal
    FE->>FE: compute selection offsets inside data-chunk-body
    FE->>API: POST note with chunk_id and anchor offsets
    API->>NS: create anchored note
    NS->>DB: validate space, pin, chunk, offsets and insert note
    FE->>API: GET annotations
    API-->>FE: anchored notes ordered by chunk ordinal and start offset
    FE->>FE: paint clipped highlight ranges
```

8. Data architecture

Diagram (mermaid source)

```
erDiagram
    users ||--o{ research_spaces : owns
    research_spaces ||--o{ pins : contains
    papers ||--o{ pins : pinned
    papers ||--o{ chunks : split_into
    research_spaces ||--o{ turns : has
    turns ||--o{ citations : cites
    chunks ||--o{ citations : cited_by
    turns ||--o{ verification_iterations : audited_by
    research_spaces ||--o{ notes : has
    papers ||--o{ notes : annotates
    chunks ||--o{ notes : anchors
    papers ||--|| paper_profiles : profiled_by
    research_spaces ||--o{ comparison_reports : has
    research_spaces ||--|| space_memory : summarizes
```

Storage choices: SQLite is used for the delivered local development application; embeddings are stored in `chunks.embedding_json` so the system can rehydrate without Qdrant; `InMemoryVectorStore` provides runtime cosine

search; `verification_iterations` stores audit records; notes store optional paper, chunk, quote, offsets, and color.

9. Key design decisions and trade-offs

Decision	Rationale	Trade-off
Local Ollama for LLM and embeddings	Preserves privacy and avoids hosted LLM cost.	Latency and quality depend on local hardware and small models.
Plain async verification loop instead of LangGraph	Faster to implement and easier to test as normal Python.	No graph checkpointing or visual orchestration.
SQLite plus in-memory vector store	Zero-service local quickstart and deterministic tests.	Not suitable for high concurrency or large corpora.
Store embeddings in SQLite JSON	Allows rehydration after restart without vector DB.	Slow and space-inefficient at scale.
Hard per-source <code>asyncio.timeout</code>	Keeps slow APIs from blocking fan-out.	Some slow-but-valid responses are discarded.
Semantic Scholar requires API key	Avoids unauthenticated rate-limit failures.	Search can appear degraded until configured.
pypdf page extraction instead of GROBID	Simple local dependency and page numbers.	No layout-aware scholarly structure.
ContextBuilder word budget of 1800	Prevents Ollama context overflow and empty drafts.	Some relevant chunks are omitted.
Best-effort finalizer strips unsupported claims	Safer than returning a fully rejected draft.	Answers may be partial.

10. Scalability, performance, reliability, security, and privacy

Current scale target is local or demo use. The main bottlenecks are local model latency, SQLite write serialization, and in-memory vector search. The architecture can evolve by swapping SQLite for Postgres, replacing `InMemoryVectorStore` with Qdrant, and moving ingestion to a worker queue, but those are not implemented.

Performance controls include concurrent source fan-out, hard per-source deadlines, identifier-indexed dedupe, embedding batches of 32, dense and sparse hit limits, rerank and MMR limits, and an 1800-word context budget.

Reliability controls include circuit breakers, source health returned to the UI, SQLite locked retries, explicit ingest states, and verification fallbacks for timeout and `LLMUnavailableError`.

Security and privacy controls include public-HTTPS URL validation, redirect revalidation, 30 MB PDF cap, 5 MB web capture cap, PDF magic-byte checks, robots.txt checks for full web capture, untrusted evidence delimiters in prompts, and local-only LLM inference.

11. Capacity and latency characteristics

Operation	Characteristic
Scholarly search	About 4 to 13 seconds depending on public API throttling and degraded sources.
Chat with verification	About 20 to 90 seconds on local models, longer if the wall-clock cap is raised.
Paper ingest	About 6 to 17 seconds per paper, depending on PDF download, parsing, and embedding.
Deduplication	Optimized from $O(n^2)$ behavior to about 2 ms for typical fan-out batches.
Context packing	Bounded to prevent empty drafts caused by oversized local model prompts.

12. Design vs. implementation

The old `docs\01-requirements.md`, `02-architecture.md`, `03-file-structure.md`, and `04-system-design.md` were design-phase documents. The delivered implementation differs in these important ways:

Design-phase claim	Actual delivered system
LangGraph orchestrates the QA graph.	No LangGraph is wired. <code>VerificationLoopDriver</code> is a plain async loop.
Celery and Redis run ingestion jobs.	Ingestion runs in-process inside API requests. Redis is configured only as a setting.
Qdrant stores embeddings.	Qdrant has a health stub, but the working path stores embeddings in SQLite JSON and loads <code>InMemoryVectorStore</code> .
Postgres is the primary database.	SQLite <code>backend\data\dev.db</code> is the actual default.
GROBID parses PDFs.	PDFs are parsed with <code>pypdf</code> ; GROBID is only a setting.
Production auth and multi-tenancy.	Not implemented. A <code>users</code> table exists but is not used by routes.
Cross-encoder reranker.	Reranking is a lightweight lexical overlap heuristic.
Exact-paper lookup route.	Interface hooks exist, but no live route implements exact lookup.
Alembic migrations.	Not wired. Startup uses <code>create_all</code> and idempotent column additions.